

DhanSetu

Product Story & Key Performance Indicators

Prepared for executive and investor review

DHANSETU — the story, table by table

This is the narrative version of `database/SCHEMA.md`: not “what does this table hold” but “**what happens, in what order, and who gets value from it.**” Told through two real rows in the demo dataset — Lakshmiben Patel (merchant) and Prakash Nair (her field officer) — because following one enterprise end-to-end is more honest than describing the system in the abstract. Every number below is real, queryable, and cited in `database/data/dhansetu_v1_2/SLIDE_CORRECTIONS.md`.

The problem, in one sentence

A dairy unit in Anand is profitable across the year and still collapses every April–September, when heat cuts milk yield at the exact moment fodder costs peak. A bank statement doesn’t show that coming. A bureau score doesn’t either — Lakshmiben has no bank loan to default on, so a default-probability model reads her as risk-free right up until she’s borrowing from a moneylender to stay afloat. **DHANSETU exists to see the squeeze before the missed payment, and to answer it with money instead of a rejection.**

Part 1 — the merchant’s side: Lakshmiben Patel, ENT0031

Lakshmiben Gopal Dairy, Anand, Gujarat. Dairy producer, 38, shares a device with family, settles with her co-op every ~14 days. Logs in with her phone number in the mobile app.

1. Every day, without her doing anything extra

Her co-op payments and expenses land in `daily_ledger` — one row per day: cash in, cash out, EMI due/paid, informal debt, digital payment share. She doesn’t fill out a form for this; it’s the transaction trail her business already produces. (`merchant_entries` exists for the version of this that *does* need her voice — see Sunita’s story below.)

Value to her: none yet, directly — this is the raw material. The value starts at step 2.

2. Once a month, the system looks back and asks “how exposed is she right now”

`feature_snapshots` computes ~70 point-in-time indicators as of each month end: margin gap over the last 90 days, buffer days, DSCR, receivable ageing, how much of her buffer is actually her own money vs. borrowed. For 2026-07-31: cost index **+21.5%** over three months (fodder), procurement price **–0.2%** (milk) — a **21.7 percentage-point margin gap**. This is the “feed index up 14% while milk price stayed flat” story, as an actual number tied to her ID.

Value to her: the system is watching the thing that actually hurts her — the *gap* between her cost curve and her revenue curve — not a generic “missed payment” trigger that would fire too late to help.

3. The gap gets scored, fused, and given a tier

`risk_assessments` combines a model score (probability of *stress*, not default — deliberately, see below) with a rule engine (`rules`, `rule_evaluations`) into one `fused_score` and a tier: GREEN / AMBER / RED. Lakshmiben is **AMBER**, `net_buffer_days = -12`.

That `net_buffer_days` figure matters more than the tier itself: her *gross* buffer is 12 days, but netted against what she owes moneylenders, she’s actually **57 days underwater**. Under an old, cruder version of this model (scored on default probability), she came out **GREEN** — because she has no bank loan to default on. Reading “no formal debt” as “no risk” is exactly the blind spot that makes bureau-based lending miss borrowers like her.

Value to her: her real financial position — including the informal debt a bureau can’t see — determines her tier, not the absence of a formal loan.

4. The system explains *why*, not just *that*

`v_fired_rules` and the `reason_1/2/3` columns give three ranked, named causes: **margin_squeeze** → **working_capital_erosion** → **debt_overhang**. Not “risk score 0.62” — an actual mechanism, drawn from a closed vocabulary of six (`mechanisms`) shared by the rules, the reason codes, and the ground-truth labels (`stress_episodes`), so the explanation can later be checked against what actually happened to her, not just asserted.

Value to her (and her officer): an alert that says *what to do about it* requires knowing *why* first. “Margin squeeze” points at prebooking feed and a bridge loan; “debt overhang” alone would point at debt restructuring instead. Getting the mechanism right changes the recommendation.

5. The system forecasts a band, not a number, and finds the shortfall

`forecasts` → `v_live_forecast` gives her a forward p10/p50/p90 cash path at six horizons (30–180 days). `v_projected_shortfall` finds the deepest point of her downside path: **–₹31,000 in the week of 29 August 2026** (her 90-day p10 trough is –₹46,739 on 29 September). Measured out-of-time accuracy at that horizon: **MAE ₹39,104, 77.6% of actuals inside the 80% band** — numbers the system is willing to publish about itself, not just claim.

Value to her: she knows the shape of the problem — how bad, and *when* — early enough to act, instead of finding out the day the payment bounces.

6. The alert arrives with an answer, not just a warning

`alerts` (raised **2026-06-30**, tier history: RED at April, AMBER since) is paired via `recommendations` → `v_alert_actions` with up to three ranked, mechanism-matched actions in *her* language (Gujarati): `prebook_input`, `request_bridge_loan`, `collect_udhaar`.

The headroom numbers are the point of the whole product: **term credit headroom ₹0** (a normal loan officer would stop here) but **bridge headroom ₹21,600** — a short-tenor facility engineered to stay positive precisely in distress, driven by cash flow/uncertainty/behaviour rather than by her tier. Across the whole book, tier explains only **7.7%** of the variance in headroom, and AMBER/RED headroom ranges overlap with GREEN’s — proof this isn’t tier-with-extra-steps.

Value to her: risk monitoring becomes origination. The system’s answer to “you’re in trouble” is a specific amount of money she can access, not a rejection letter.

7. `alerts.exported_to_bureau = false`, always

Every single row. The “your worst month is perishable information, not a permanent scar” promise is a column value she (or an auditor) can check, not a slide claim.

Value to her: a bad quarter during the dairy lean season doesn't follow her to the next lender.

8. She controls who sees what

`consent_artifacts` (her grants, time-boxed, revocable), `access_grants` (who can see her data, at which tier, until when), `audit_log` (every access tied to a grant, `merchant_notified = true`), `daily_ledger` and `merchant_entries` carry `is_household`, so purely personal cash movements are structurally excluded from business risk scoring, not just policy-excluded.

Value to her: she can see, in principle, exactly who looked at her data and under what permission — and household spending never becomes a business red flag by construction.

Part 2 — the officer's side: Prakash Nair, FO1, Anand

42 enterprises, exactly — every officer covers one district's worth, no one spans two states. Logs into the same backend, different token (`role: officer`), different worklist.

1. He doesn't review 42 businesses — he reviews the ones that need him

`v_officer_worklist` filters to enterprises assigned to him that are AMBER or RED, ranked by score descending. In practice this is a small fraction of his caseload (see `demo_queries.sql` §1b) — the tool's first job is attention triage, not surveillance of everyone all the time.

Value to him: a 42-name caseload becomes a short, ranked shortlist. He opens the app and knows where to go first.

2. The list is ranked by rupees at risk, not just a score

`v_officer_worklist` surfaces `rupees_at_risk` (the projected shortfall from the linked alert) alongside the abstract `score` — for Lakshmiben, ₹31,000 with a deadline of **2026-08-29**. When every name on his list shares one underlying cause (see District Event Watch below), sorting by rupees is the sort that actually matters.

Value to him: he can answer “why her, why now, why urgent” in one glance, not by opening each case.

3. Each name comes with a route, not just an address

`km_from_centre` (straight-line distance from the district geographic centroid, via `district_geo`) lets his list double as a rough route plan. Small detail, but it's the difference between a list and a day plan.

4. He opens her card and gets the full “why,” not a bare score

`GET /enterprise/{id} → v_enterprise_card + v_fired_rules + v_live_forecast` gives him everything from Part 1: tier, reason codes, margin gap, forecast band, headroom — the same explanation Lakshmiben's app shows her, so the conversation in her yard matches what she's already seen, not a second, unexplained number.

Because he's an **officer**, not a merchant, he can pull *upany* enterprise in the system, not just his own caseload — needed for the next case.

5. District Event Watch: one shock, not fourteen identical alerts

When a single event (a price crash, a cyclone) hits a whole district, `v_district_event_watch` collapses what would otherwise be 14 separate AMBER alerts with the same `reason_1` into **one district-level finding**: percentage of the cohort affected, whether it crosses the 30% threshold that marks it a genuine “district event” (rather than 14 individual business problems), and the three worst-off enterprises to visit first (`worst_first[1:3]`, ordered by buffer days).

Value to him: he doesn't waste a day treating a weather event as 14 unrelated cases — he sees it's one event and knows which three people to see first because they have no buffer left, not because their score happens to be marginally higher.

6. He visits, and closes the loop

`POST /outcome → record_outcome()` takes his visit result — `outcome` (`stress_confirmed` / `false_positive` / `unreachable`), the `intervention` he actually offered (e.g. `request_bridge_loan`) — and: - writes `visit_outcomes` (defaulting the note language to *her* preferred language, not his, since this becomes a record about her), - closes his `officer_tasks` row, - flags the row `becomes_training_label = TRUE`.

Value to him: filing the visit outcome is a two-field form, not paperwork — and it's the same action that makes the whole system smarter next month.

7. The system tells on itself: was AMBER worth his petrol?

`v_alert_precision` reports confirmation rate by tier from real visit outcomes — of the alerts he and every other officer acted on, what percentage were confirmed stress vs. false positive. This is the honesty check on his own worklist: if AMBER confirmation rate is weak, that's a signal to retune the model, not a fact to hide from him.

Value to him: he isn't asked to trust a black box indefinitely — the system reports its own hit rate on the exact thing he's spending fuel and time on.

Two more merchants, for the edge cases the happy path hides

- **Sunita Devi, ENT0067 (Bhilwara, pottery, IVR channel, `low_visibility = true`)**. Her digital footprint is thin (`digital_share` 0.06 → 0.09), so she calls in instead of tapping a phone. `voice_entries` + `voice_extractions` capture the call and the parsed amount; `needs_review` (`v_voice_review_queue`) flags anything with low ASR confidence or a missing amount for a human to check before it becomes a ledger fact via `v_daily_from_voice`. `low_visibility = true` on her risk assessment is itself an honest signal — the system telling Prakash “*my confidence in this read is lower than usual*,” rather than quietly guessing. **Value:** the merchants hardest to see on paper are exactly the ones the product is for — the pipeline is built to admit uncertainty about them, not paper over it.
- **Basanti Pradhan, ENT0224 (Ganjam, kirana store, cash-dominant, assisted-channel)**. She can be **GREEN** on the standard risk tier while quietly bleeding lakhs to bad udhaar (`v_receivables_ageing`: high `write_off_pct`, long `avg_days_to_cash`). Tier alone would miss this — it's a separate view precisely because “receivables health” and “cash buffer health” are different failure modes that don't always move together. **Value:** the receivables book gets

its own lens instead of being averaged away inside a single score.

What this adds up to

Who	Value delivered
Merchant	Sees a squeeze coming (not a missed payment already happened); gets <i>areason</i> , not a raw score; gets an actual <i>credit answer</i> (bridge headroom) instead of a rejection when the tier goes red; a bad season doesn't get exported to a bureau; controls and can audit who sees her data.
Field officer	A 42-name caseload becomes a short, ranked, routed shortlist; district-wide shocks read as one event, not N duplicate alerts; every recommendation comes with the <i>why</i> , matching what the merchant already sees; closing a visit is a two-field form that simultaneously produces tomorrow's training label.
The institution / evidence layer	Every claim on the pitch deck is a live query, not a slide number: reason-code accuracy (<code>v_reason_code_scorecard</code>), lead time (<code>v_lead_time_summary</code>), forecast accuracy (<code>v_forecast_accuracy</code>), alert precision (<code>v_alert_precision</code>), headroom-vs-tier independence (<code>v_headroom_by_tier</code>) — so the system's own honesty about where it's weak (9 lead-time episodes, 15 missed-repayment positives) is part of the product, not a gap hidden from the judges.

One loop, closing on itself

`daily_ledger/merchant_entries` → `feature_snapshots` → `risk_assessments` → `alerts` → `officer_tasks` → `visit_outcomes` (→ back into training data for the next `risk_assessments` run) is the entire spine. Every table in [database/SCHEMA.md](#) exists to move an enterprise through that loop faster, more explainably, or more honestly than a bureau-score-and-reject model would.

DHANSETU — KPIs, and where they come from

Every KPI below traces to a real column, table, or view — no metric here is aspirational. Read alongside [STORY.md](#) (the narrative these numbers serve) and [Database/SCHEMA.md](#) (the full schema reference).

Two audiences, two different questions: - **Merchant KPIs** answer “where do I stand, and what can I do about it?” - **Officer KPIs** answer “who do I see today, and is this tool worth my time?”

API column key: **LIVE** = a live route returns this today (see [API.md](#)). **PENDING** = the DB view/column exists but no route reads it yet.

1. Merchant-facing KPIs

KPI	Definition	Source	API	What it tells the merchant
Risk tier	GREEN / AMBER / RED	<code>risk_assessments.risk_tier</code> (← <code>v_enterprise_card</code> , <code>v_officer_worklist</code>)	LIVE <code>GET /enterprise/{id}</code>	One-glance status. Drives everything downstream — but see “Credit headroom” below for why tier ≠ verdict on credit.
Net buffer days	Gross buffer, netted against informal debt	<code>feature_snapshots.net_buffer_days</code> / <code>risk_assessments.net_buffer_days</code>	LIVE <code>GET /enterprise/{id}</code>	The honest runway number. Lakshmben: 12 days gross, -57 net the gap a bureau-based score can't s
Cash flow forecast band	p10/p50/p90 forward cash path, 30–180 days	<code>v_live_forecast</code>	LIVE <code>GET /enterprise/{id}</code>	Not “will I have money” but “here's th range, and how wide it is” — <code>band_width</code> (<code>risk_assessments.band_width</code>) k itself a KPI: a wide band means low confidence, not good news.
Projected shortfall	Deepest point of the p10 downside path, and the week it lands	<code>v_projected_shortfall</code> (<code>amount</code>), <code>alerts.shortfall_week_of</code> / <code>deadline_date</code>	LIVE <code>GET /enterprise/{id} (latest_alert)</code>	The number with a deadline attached turns a forecast into something actionable before the fact.
Margin gap (90d)	Cost index change – revenue index change	<code>feature_snapshots.margin_gap_90d</code> , <code>cost_index_chg_90d</code> , <code>rev_index_chg_90d</code>	LIVE <code>GET /enterprise/{id}</code>	The mechanism, quantified — Lakshmben's 21.7pp gap is <i>why</i> she' AMBER, not just <i>that</i> she is.
Credit headroom / bridge headroom	Facility size the merchant can access right now — term vs. short-tenor	<code>risk_assessments.credit_headroom</code> , <code>bridge_headroom</code> , <code>suggested_max_eml</code>	LIVE <code>GET /enterprise/{id}</code> , <code>GET /risk/{id}/predict</code>	The KPI that makes this a lending product, not a warning system. Deliberately not derived from tier — variance explained by tier is only 7.7% (<code>v_headroom_by_tier</code>), so AMBER doesn't automatically mean “no headroom.”
DSCR (debt service coverage)	Annualised ability to cover EMI from cash flow	<code>feature_snapshots.dscr_annual</code> , <code>risk_assessments.dscr_annual</code>	LIVE <code>GET /enterprise/{id}</code>	Repayment capacity, independent of whether a payment has actually been missed yet.
Missed EMIs (90d / 365d)	Count of missed scheduled installments	<code>feature_snapshots.missed_emis_90d</code> , <code>missed_emis_365d</code>	LIVE 90d via <code>/enterprise/{id}</code> , both via <code>GET /risk/{id}/predict 's features</code>	Lagging, but still shown — the forwar looking KPIs above exist precisely so merchant sees trouble <i>before</i> this one moves.
Receivables ageing (the udhaar book)	Outstanding amount, write-off %, avg/worst days-to-cash, by counterparty	<code>v_receivables_ageing</code>	LIVE <code>GET /enterprise/{id}/receivables</code>	Basanti Pradhan's case: this can be bleeding money while the risk tier stay GREEN — it's a separate KPI because it's a separate failure mode.
Digital payment mix	UPI share / wallet share / cash share, full panel and trailing 90d	<code>v_merchant_payment_mix</code>	LIVE <code>GET /enterprise/{id}/payment-mix</code>	Both a literacy/formalization signal and a leading indicator: a rising cash shan alongside a falling <code>digital_share_slope</code> (<code>enterprises.digital_share_slo</code>) often precedes reduced data visibility
Data completeness / low visibility	Confidence flag on the merchant's own score	<code>feature_snapshots.data_completeness</code> , <code>risk_assessments.low_visibility</code>	LIVE <code>GET /enterprise/{id}</code>	Honesty as a KPI: Sunita Devi's <code>low_visibility = true</code> tells her (Prakash) the system's confidence is lower than usual, instead of quietly guessing.

KPI	Definition	Source	API	What it tells the merchant
Reason codes 1–3	Top 3 named mechanisms behind the score, with contribution weight	<code>risk_assessments.reason_1/2/3 + _contrib, v_fired_rules</code>	LIVE codes via <code>/enterprise/{id}</code> ; PENDING <code>v_fired_rules</code> (per-rule weight detail) has no route	Explainability as a KPI, not just a UI nicety — this is what makes reason codes independently scorable (see <code>v_reason_code_scorecard</code> , section now also live).
Alert status	Raised date, deadline, expiry, dispute state, bureau-export flag	<code>alerts.raised_at/deadline_date/expires_at/disputed_at/exported_to_bureau</code>	LIVE <code>GET /enterprise/{id} (latest_alert)</code>	<code>exported_to_bureau</code> is <code>false</code> on every row, always — the “perishable, not permanent” promise as a checkat value.
Consent & access transparency	Who has accessed my data, under what grant, was I notified	<code>consent_artifacts, access_grants, audit_log.merchant_notified</code>	PENDING no route at all	A trust KPI, not a risk KPI — “can I see who’s looked at my data” matters as much as any financial number to the target user.

2. Officer-facing KPIs

KPI	Definition	Source	API	What it tells the officer (or their supervisor)
Worklist size vs. caseload	Count of AMBER/RED enterprises + total assigned	<code>v_officer_worklist.count</code> vs. <code>officers.caseload</code> (see <code>demo_queries.sql</code> §1b, <code>pct_reviewed</code>)	PENDING the worklist itself is returned; the coverage % is not computed by any route	Attention triage, quantified — proof the tool narrows 42 names to a short list rather than asking him to review everyone.
Rupees at risk	Latest alert’s projected shortfall, per worklist row	<code>v_officer_worklist.rupees_at_risk</code> (from <code>alerts.projected_shortfall</code>)	LIVE <code>GET /worklist</code>	The sort key that matters when many names share one cause — money at risk, not an abstract score.
Fused score	Model + rule-engine blended risk score	<code>risk_assessments.fused_score / v_officer_worklist.score</code>	LIVE <code>GET /worklist</code>	Secondary ranking signal — used together with rupees-at-risk, not instead of it.
Route distance	Straight-line km from district centroid	<code>v_officer_worklist.km_from_centre</code> (via <code>district_geo</code>)	LIVE <code>GET /worklist</code>	Turns a list into a rough day plan.
District event flag	Whether ≥30% of a district-sector cohort is flagged under one mechanism	<code>v_district_event_watch.is_district_event, pct_of_cohort</code>	LIVE <code>GET /evidence/district-events</code>	Tells the officer “this is one shock, not 14 separate problems” — and hands him the 3 worst-off (<code>worst_first[1:3]</code>) to visit first.
Task turnaround	Time from <code>officer_tasks.assigned_on</code> to <code>visit_outcomes.visited_on</code>	<code>officer_tasks, visit_outcomes</code>	PENDING no view, no route	Operational SLA — how fast alerts actually get worked, not just raised. (<i>Still an open gap, see below.</i>)
Outcome mix	Share of visits that are <code>stress_confirmed / false_positive / unreachable</code>	<code>visit_outcomes.outcome</code>	PENDING <code>POST /outcome</code> writes it, nothing reads the aggregate back	Is he finding real problems, or chasing noise, or unable to reach people at all (a different problem — access, not risk).
Alert precision by tier	% of visited AMBER/RED alerts confirmed as real stress	<code>v_alert_precision.confirm_pct</code>	LIVE <code>GET /evidence/alert-precision</code> (book-wide, not per-officer — see gaps)	The self-honesty KPI: “is AMBER worth the officer’s petrol?” If confirmation rate is weak, that’s a signal to retune the model, not a fact to hide from him.
Reason-code accuracy	% of episodes where the predicted top-1 / top-3 reason code matches the true mechanism	<code>v_reason_code_scorecard.top1_pct/top3_pct</code>	LIVE <code>GET /evidence/reason-code-scorecard</code>	Whether he can trust the <i>explanation</i> he’s about to repeat to the merchant, not just the tier.
Early-warning lead time	Days between first flag and first missed repayment, and % of episodes caught at all	<code>v_lead_time_summary.median_lead_days, caught</code>	LIVE <code>GET /evidence/lead-time</code>	Whether the whole system gives him enough runway to act — median 121 days, but n = 9 out-of-time episodes , an honest caveat worth keeping attached to this KPI rather than quoting it bare.
Training-label contribution	Count of visit outcomes marked <code>becomes_training_label = TRUE</code>	<code>visit_outcomes.becomes_training_label</code>	PENDING no route	Filing an outcome isn’t just paperwork — it’s a measurable contribution to next month’s model.
Language/channel match rate	Officer’s language vs. merchant’s <code>preferred_lang/preferred_channel</code>	<code>officers.language</code> vs. <code>enterprises.preferred_lang/preferred_channel</code>	PENDING no view, no route	Quality-of-visit signal, not just logistics — a mismatch predicts a worse conversation before it happens.

3. Shared / evidence-layer KPIs (both sides benefit, judges/lenders watch these too)

These are less “your daily number” and more “why should anyone trust this system’s other numbers” — already the backbone of the evidence views in database/SCHEMA.md, and now their own endpoint group:

KPI	Source	API
Forecast accuracy (MAE, p10–p90 coverage) by horizon	v_forecast_accuracy	LIVE GET /evidence/forecast-accuracy
Headroom independence from tier (proves headroom ≠ tier restated)	v_headroom_by_tier	LIVE GET /evidence/headroom-by-tier
Observed vs. hand-authored seasonality	v_sector_seasonality_observed	PENDING no route
Data provenance (real vs. simulated share, per enterprise)	v_data_provenance	LIVE GET /evidence/data-provenance

Gaps worth naming, not hiding

- **No dedicated task-turnaround view.** officer_tasks.assigned_on → visit_outcomes.visited_on is currently a join a caller has to write themselves, not a rollup like v_alert_precision. Same shape of fix as v_merchant_payment_mix was for payment mix, if this becomes a KPI worth showing in the app.
- **No link from a recommendation to an actual disbursed loan.** recommendations.action_key (e.g. request_bridge_loan) records what was suggested; there's no FK from a recommendation or outcome to a loans row, so "did the bridge loan actually get disbursed" isn't queryable yet — it lives only as visit_outcomes.intervention (free text), not a structured conversion KPI.
- **v_alert_precision is book-wide, not per-officer.** It answers “is AMBER worth the petrol” in aggregate; a per-officer breakdown would need a GROUP BY officer_id added via officer_tasks, which the current view doesn't join to.